

claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a091336afdd0f509f.jsonl`  
- SHA-256: `fa3ead065badb4ca21aced7c1174283557b261918db74b5bbeed5acaaf49b7cb`  
- Source modified: `2026-07-06T17:12:03+00:00`  
- Imported at: `2026-07-08T15:59:36+00:00`  
- project: `subagents`  
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`
```

Transcript

1. user (2026-07-06T17:11:02.462Z)

Read-only investigation in the repo at C:\Users\avidu\Projects\khesutra-guru\badminton-highlight-indexer. I need to understand how the Cloud Run Job named "rally-worker" picks which video to process and how a run is triggered, so that I can trigger my OWN test runs on a specific (downsampled) video I upload to GCS.

Evidence from a real execution's logs: the worker mounts gs://khesutra-rally-serving via GCSFuse, pulls a video from gs://khesutra-rally-serving/videos/uploads/<uploadId>/<name>.MP4 into /tmp, runs detection, writes outputs to gs://khesutra-rally-serving/outputs/<videoId>/{intervals.csv,report.json}, and writes a completion marker gs://khesutra-rally-serving/_dispatch/<dispatchId>.done.

Read: backend/worker.py (main entrypoint / how it learns the video to process), deploy/cloudrun/(gen_service_yaml.py, cloudbuild.yaml, Dockerfile, README.md), and grep the repo for: "_dispatch", "dispatch", "rally-worker", "jobs execute", "run jobs", "execution", "medium_uri", "video_uri", "uploads/", "RALLY_", env var names the worker reads. Also check backend for the control-plane side (rally-control-plane service) that enqueues work, and any code that calls `gcloud run jobs execute` or the Cloud Run Admin API to launch an execution.

Answer CONCRETELY with file:line and exact commands/formats:

1. How does a rally-worker execution learn WHICH video to process ? env var, container args, a dispatch file in gs://.../_dispatch/, or a DB/queue row? What is the exact payload/format (field names)?
2. How is an execution actually launched? (e.g. `gcloud run jobs execute rally-worker --region=... --update-env-vars=...` or `--args=...`, or triggered by the control plane / a GCS notification?) Quote the exact mechanism.
3. What env vars / config does the job read at runtime (e.g. WASB_WEIGHTS, RALLY_CONFIG, a video-URI var, timeout_sec, skip_ai_handoff, a proxy/downsample flag)?
4. Give me the concrete recipe to run the SAME rally-worker container on a DIFFERENT video I upload to gs://khesutra-rally-serving/videos/uploads/<newId>/<name>.MP4 ? i.e. exactly what to upload and what command/trigger makes rally-worker process it, and where its outputs will land.

Quote code/scripts. Do not modify or launch anything. Return a concise, actionable findings report with the exact trigger command.

2. assistant (2026-07-06T17:11:04.903Z)

I'll investigate the codebase to understand the rally-worker job mechanics, the dispatch mechanism, and how to trigger it on a custom video.

3. user (2026-07-06T17:11:05.234Z)

No files found

4. user (2026-07-06T17:11:05.477Z)

deploy\cloudrun\gen_service.yaml.py

5. user (2026-07-06T17:11:05.504Z)

deploy\cloudrun\cloudbuild.yaml

6. user (2026-07-06T17:11:05.668Z)

deploy\cloudrun\Dockerfile

7. user (2026-07-06T17:11:05.685Z)

No files found

8. assistant (2026-07-06T17:11:07.455Z)

Now let me read the main worker.py and deployment files to understand the mechanics:

9. user (2026-07-06T17:11:08.787Z)

```
1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
```

```

33     # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
34     # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
35     # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
36     # MAMBA_ROOT_PREFIX=$CONDA_DIR (= /opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
37     # is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are
unaffected by the switch.
38     ENV MAMBA_ROOT_PREFIX=/opt/conda
39     RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
40         | tar -xj -C /usr/local bin/micromamba \
41         && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
42         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
43         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
44             torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
45         && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
46         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
47             hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
48
49     # --- Main application env ---
50     WORKDIR /app
51     COPY pyproject.toml README.md ./
52     COPY requirements-trackers.txt ./
53     COPY backend ./backend
54     COPY training ./training
55     # Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker
PULLS the gs://
56     # input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-
storage is
57     # lazy-imported there); without it the job dies at the first gs:// fetch with
`ModuleNotFoundError`:
58     # google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE,
which dispatches
59     # the L4 worker Job via google.cloud.run_v2 (backend/compute/cloud_run.py, lazy-
imported); without it
60     # the control plane fails every dispatch with "google-cloud-run is required for
CloudRunCompute's real
61     # client". Both are real Cloud Run failures the mock-client tests can't see (the SDKs
are faked in CI).
62     # auth: PyJWT[crypto] for M9 edge-auth (security.edge_auth_enabled ? the Cf-Access JWT
verify the
63     # reused-as-control-plane deployment runs behind api.khelsutra.guru); jwt is lazy-
imported and
64     # edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-
plane
65     # deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT
overlay).
66     # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
67     # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
68     # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
69     # single-sourced in requirements-trackers.txt.
70     RUN python3 -m pip install --upgrade pip \
71         && python3 -m pip install -r requirements-trackers.txt \
72         && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
73
74     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
75     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the

```

```

default.
76      # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
77      # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
78      # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
79      ENV PYTHONUNBUFFERED=1 \
80          WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
81          WASB_REPO_DIR=/opt/models/WASB-SBDT \
82          WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
83          RALLY_DETECTOR_IMPL=native
84
85      # The dispatcher (CloudRunCompute) supplies the per-execution args:
86      # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
87      ENTRYPOINT ["python3", "-m", "backend.worker"]
88

```

10. user (2026-07-06T17:11:09.047Z)

```

1      """Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING
M8/M9, #473).
2
3      The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in
a
4      session scratchpad ? this commits the generator so the deploy is reproducible from a
clean
5      clone. It emits a Knative service manifest for::
6
7          python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
8          gcloud run services replace service.yaml --region asia-southeast1
9
10     Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container
command is
11     a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows
the gcloud
12     cmd-shim mangles those through `--command/--args`, while a YAML round-trips them
verbatim. Run
13     gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).
14
15     The app is configured through a config.json baked INTO the manifest: the wrapper
base64-decodes
16     it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before
exec'ing the
17     server, so one image serves every config shape (config.json wins over the cloud-serving
preset ?
18     backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
19     identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9)
before the
20     service faces testers again.
21
22     Deliberate manifest choices (learned on the A1 deploy):
23     * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job
worker drains
24     on a background thread; without always-allocated CPU a 202'd compile stalls (README
§3a).
25     * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a
second
26     instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise
it only
27     once state is shared.
28     * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is
rejected
29     with it; the code default ("rally-worker") is already correct.

```

```

30      * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers
the
31      committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
32      `skip_ai_handoff=true` and never needs it).
33
34      Baked config pins (CONTROL_PLANE_DURABLE_REBUILD P1/D5 ? the live rev-00011 posture,
learned on
35      the 2026-07-05 owner CUJ; before this, regenerating the manifest silently DROPPED both):
36      * `indexing.default_segmenter=wasb_hybrid` ? the engine default is yolo_hybrid, so an
unpinned
37      cloud job ran the wrong segmenter (#479).
38      * `security.max_part_mb=24` ? upload parts must fit under the ~32 MiB HTTP/1 body cap
at the
39      Google Front End (#480); the engine default (95) gets every part 413'd before
FastAPI runs.
40      NOT fixed via `use-http2`: h2c requires the container itself to speak cleartext
HTTP/2,
41      which uvicorn does not (spec D6) ? the annotation would break the service.
42      * `--memory` defaults to 8Gi (spec D2) ? compile still pulls the gs:// source into the
43      RAM-backed /tmp to stitch, so the instance must hold the largest allowed upload with
44      headroom; >8Gi would force 4 vCPU on Cloud Run for no current need.
45      ""
46
47      from __future__ import annotations
48
49      import argparse
50      import base64
51      import json
52      import sys
53
54      DEFAULT_PROJECT = "gen-lang-client-0306026826"
55      DEFAULT_REGION = "asia-southeast1"
56      DEFAULT_SERVICE = "rally-control-plane"
57      DEFAULT_SERVING_BUCKET = "gs://khesutra-rally-serving"
58      DEFAULT_CF_TEAM_DOMAIN = "https://khesutra.cloudflareaccess.com"
59      DEFAULT_CF_ACCESS_AUD =
"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"
60
61
62      def build_config(args: argparse.Namespace) -> dict:
63          ""The app config the wrapper materializes at RALLY_APP_HOME/config.json.""
64          bucket = args.serving_bucket.rstrip("/")
65          return {
66              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
67              "indexing": {
68                  # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this
service.
69                  "skip_ai_handoff": True,
70                  # rev-00011 pin (#479): the engine default is yolo_hybrid ? pin the
validated one.
71                  "default_segmenter": args.default_segmenter,
72              },
73              "storage": {
74                  "backend": "gcs",
75                  "output_root": bucket,
76                  "input_backend": "gcs",
77                  "input_root": f"{bucket}/videos",
78              },
79              "security": {
80                  "edge_auth_enabled": args.edge_auth == "on",
81                  "cf_team_domain": args.cf_team_domain,
82                  "cf_access_aud": args.cf_access_aud,
83                  "allowed_video_root": "/tmp/apphome/output",
84                  # rev-00011 pin (#480): parts must clear the ~32 MiB HTTP/1 cap at the GFE.
85                  "max_part_mb": args.max_part_mb,

```

```

86         },
87     }
88
89
90 def build_yaml(args: argparse.Namespace) -> str:
91     """Render the Knative service manifest for `gcloud run services replace`. """
92     cfg_b64 = base64.b64encode(
93         json.dumps(build_config(args), separators=(",", ":")).encode("utf-8")
94     ).decode("ascii")
95     # mkdir BEFORE the redirect (the redirect can't create the directory), then exec so
96     # server is PID 1 and Cloud Run's SIGTERM reaches it directly.
97     wrapper = (
98         "mkdir -p /tmp/apphome && "
99         f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
100         "export RALLY_APP_HOME=/tmp/apphome && "
101         "exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"
102     )
103     return f"""\
104 apiVersion: serving.knative.dev/v1
105 kind: Service
106 metadata:
107   name: {args.service}
108   labels:
109     cloud.googleapis.com/location: {args.region}
110 spec:
111   template:
112     metadata:
113       annotations:
114         autoscaling.knative.dev/minScale: "{args.min_instances}"
115         autoscaling.knative.dev/maxScale: "{args.max_instances}"
116         run.googleapis.com/cpu-throttling: "false"
117     spec:
118       timeoutSeconds: {args.timeout}
119       containers:
120       - image: {args.image}
121         command: ["/bin/bash"]
122         args: ["-c", "{wrapper}"]
123         ports:
124         - containerPort: 8080
125       resources:
126         limits:
127           cpu: "{args.cpu}"
128           memory: {args.memory}
129       env:
130       - name: DEPLOYMENT_PROFILE
131         value: cloud-serving
132       - name: CLOUD_RUN_REGION
133         value: {args.region}
134       - name: GOOGLE_CLOUD_PROJECT
135         value: {args.project}
136       - name: RALLY_ALLOWED_HOSTS
137         value: ""
138 """
139
140
141 def main(argv: "list[str] | None" = None) -> int:
142     p = argparse.ArgumentParser(description=__doc__.splitlines()[0])
143     p.add_argument(
144         "--edge-auth",
145         choices=("on", "off"),
146         required=True,
147         help="REQUIRED and explicit: 'off' only for the identity-token smoke/e2e window;
148         "
149         "redeploy 'on' (M9 Cf-Access verify) before testers touch it again.",

```

```

149     )
150     p.add_argument("--project", default=DEFAULT_PROJECT)
151     p.add_argument("--region", default=DEFAULT_REGION)
152     p.add_argument("--service", default=DEFAULT_SERVICE)
153     p.add_argument(
154         "--image",
155         default=None,
156         help="Container image; default = the project's rally-worker:latest (post-#470 it
157         "carries cloud-run + auth, so the worker image IS the control-plane image).",
158     )
159     p.add_argument("--serving-bucket", default=DEFAULT_SERVING_BUCKET)
160     p.add_argument("--cf-team-domain", default=DEFAULT_CF_TEAM_DOMAIN)
161     p.add_argument("--cf-access-aud", default=DEFAULT_CF_ACCESS_AUD)
162     p.add_argument(
163         "--default-segmenter",
164         default="wasb_hybrid",
165         help="rev-00011 pin (#479): the engine default is yolo_hybrid; the alpha serves
166         the "
167         "validated wasb_hybrid.",
168     )
169     p.add_argument(
170         "--max-part-mb",
171         type=int,
172         default=24,
173         help="rev-00011 pin (#480): upload parts must fit under Cloud Run's ~32 MiB
174         HTTP/1 "
175         "request-body cap (the engine default 95 gets 413'd at the edge).",
176     )
177     p.add_argument("--min-instances", type=int, default=1)
178     p.add_argument(
179         "--max-instances",
180         type=int,
181         default=1,
182         help="Default 1: per-instance SQLite state splits across instances (issue
183         #471).",
184     )
185     p.add_argument("--cpu", type=int, default=2)
186     p.add_argument(
187         "--memory",
188         default="8Gi",
189         help="Default 8Gi (CONTROL_PLANE_DURABLE_REBUILD D2): compile pulls the gs://
190         source "
191         "into RAM-backed /tmp, so size for the largest allowed upload plus headroom.",
192     )
193     p.add_argument("--timeout", type=int, default=300)
194     p.add_argument("-o", "--output", default="-", help="Output path, or '-' for
195     stdout.")
196     args = p.parse_args(argv)
197     if args.image is None:
198         args.image = f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-
199         worker:latest"
200     text = build_yaml(args)
201     if args.output == "-":
202         sys.stdout.write(text)
203     else:
204         with open(args.output, "w", encoding="utf-8", newline="\n") as f:
205             f.write(text)
206     return 0
207
208 if __name__ == "__main__":
209     raise SystemExit(main())
210

```

11. user (2026-07-06T17:11:09.300Z)

```
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\annotations\automated_provider.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\annotations\base.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\annotations\file_provider.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\annotations\__init__.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\api\auth.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\api\jobs.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\api\job_worker.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\api\uploads.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\api\__init__.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\billing.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\base.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\cloud_run.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\compute\__init__.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\config\loader.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\config\models.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\config\presets.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\config\startup.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\config\__init__.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\eval\ablation.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\eval\ablation_pdf.py
```

12. assistant (2026-07-06T17:11:10.707Z)

Now let me search for the worker.py file in the backend directory and look for dispatch mechanisms:

13. user (2026-07-06T17:11:12.086Z)

```
Found 24 files
tests\test_restart_recovery.py
tests\test_compute_backend.py
backend\worker\__main__.py
backend\orchestration.py
backend\main.py
backend\infrastructure\database_jobs.py
backend\api\job_worker.py
backend\storage\base.py
backend\storage\__init__.py
tests\test_api_cloud_dispatch.py
backend\compute\cloud_run.py
backend\compute\base.py
tests\test_submit_md5_cache.py
backend\api\jobs.py
tests\test_config_deployment.py
backend\config\models.py
backend\compute\__init__.py
tests\test_main_module_budget.py
tests\test_async_jobs.py
tests\test_drain_concurrency.py
backend\infrastructure\database.py
tests\test_worker.py
tests\test_person_detector.py
backend\worker\__init__.py
```

14. user (2026-07-06T17:11:12.440Z)

```
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\api\jobs.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\api\job_worker.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
```

```
indexer\backend\infrastructure\database_jobs.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker\__init__.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker\__main__.py
```

15. assistant (2026-07-06T17:11:14.691Z)

Great! Now let me read the key files:

16. user (2026-07-06T17:11:15.343Z)

```
1      """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2      for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4      Flow (the cloud-serving compute path):
5      1. **Dispatch** a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6          --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7      2. **Bucket-poll** `done_uri` via `execution_policy.poll_until` + the storage
layer (D16 ? the
8          completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9          it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10         cold-start is absorbed by the async poll. On a poll TIMEOUT (`max_wait_sec` can
undercut the
11         job's own `--task-timeout` budget) the shim does ONE final marker check ? a slow-
but-healthy
12         worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13         the still-running execution so the L4 doesn't keep billing until `--task-
timeout`.
14      3. **Read** the marker (it carries `video_id` + the worker's returncode) ? fetch
15         `outputs/<video_id>/report.json` ? `ComputeResult`.
16
17      The dispatched container is forced to run **INLINE** (`compute_target=local_gpu` +
native detector)
18      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19
20      The Cloud Run trigger is an **injected client** (`CloudRunJobClient`) so the whole
shim is unit-tested
21      with a fake (no GCP). The real `GoogleCloudRunJobClient` lazy-imports `google-cloud-
run` and is
22      owner-verified on the M7 real run.
23      """
24
25      from __future__ import annotations
26
27      import json
28      import logging
29      import os
30      import tempfile
31      import uuid
32      from abc import ABC, abstractmethod
33      from typing import Any, Callable, List, Optional
34
35      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36      from backend.infrastructure.execution_policy import (
37          DEFAULT_POLICY,
38          ExecutionPolicy,
39          PolicyTimeout,
40          poll_until,
41      )
```

```

42     from backend.storage import fetch, get_storage, parse_uri
43
44     LOG = logging.getLogger("compute.cloud_run")
45
46     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
47     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
48     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
49         "deployment.compute_target=local_gpu",
50         "indexing.detector_impl=native",
51     )
52
53
54     class CloudRunJobClient(ABC):
55         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
56
57         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
58         overrides the job's container ``args`` for this execution and starts it."""
59
60         @abstractmethod
61         def run_job(
62             self,
63             job_name: str,
64             argv: List[str],
65             *,
66             region: str,
67             project: Optional[str] = None,
68         ) -> str:
69             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
70             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
71
72         @abstractmethod
73         def cancel_execution(self, execution: str) -> None:
74             """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
75             times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
76             May raise ? the caller treats every failure as best-effort (the original poll
timeout is
77             NEVER masked by a cancel error)."""
78
79
80     class GoogleCloudRunJobClient(CloudRunJobClient):
81         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
82         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
83
84         def run_job(
85             self,
86             job_name: str,
87             argv: List[str],
88             *,
89             region: str,
90             project: Optional[str] = None,
91         ) -> str:
92             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
93             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable

```

```

94         # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
95         if not project:
96             raise RuntimeError(
97                 "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
98                 "to resolve the job path; set it on the control plane (see
get_compute_backend)."
99             )
100         try:
101             from google.cloud import run_v2 # type: ignore[attr-defined]
102         except Exception as exc: # pragma: no cover - real SDK not present in CI
103             raise RuntimeError(
104                 "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                 "control plane (it is intentionally NOT a CI/test dependency)."
106             ) from exc
107         client = (
108             run_v2.JobsClient()
109         ) # pragma: no cover - exercised only on the real M7 run
110         name = client.job_path(project, region, job_name)
111         overrides = run_v2.RunJobRequest.Overrides(
112             container_overrides=[
113                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
114             ]
115         )
116         op = client.run_job(
117             request=run_v2.RunJobRequest(name=name, overrides=overrides)
118         )
119         return (
120             getattr(op, "metadata", None)
121             and getattr(op.metadata, "name", "")
122             or "execution"
123         )
124
125     def cancel_execution(self, execution: str) -> None:
126         # run_job falls back to the literal string "execution" when the operation
metadata carries
127         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
128         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
129         # google-cloud-run is absent).
130         if "/executions/" not in (execution or ""):
131             raise RuntimeError(
132                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
133                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
134                 "one from the operation metadata, so there is nothing to cancel by
name)."
135             )
136         try:
137             from google.cloud import run_v2 # type: ignore[attr-defined]
138         except Exception as exc: # pragma: no cover - real SDK not present in CI
139             raise RuntimeError(
140                 "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
141                 "control plane (it is intentionally NOT a CI/test dependency)."
142             ) from exc
143         client = (
144             run_v2.ExecutionsClient()
145         ) # pragma: no cover - exercised only on a real run
146         client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
147

```

```

148
149 class CloudRunCompute(ComputeBackend):
150     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
151
152     def __init__(
153         self,
154         *,
155         run_client: CloudRunJobClient,
156         job_name: str,
157         region: str,
158         project: Optional[str] = None,
159         storage_config: Optional[dict] = None,
160         policy: ExecutionPolicy = DEFAULT_POLICY,
161         token: Optional[str] = None,
162         container_overrides: Optional[tuple[str, ...]] = None,
163     ) -> None:
164         self._client = run_client
165         self._job_name = job_name
166         self._region = region
167         self._project = project
168         self._storage_config = storage_config or {}
169         self._policy = policy
170         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
171         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
172         self._token = token
173         self._container_overrides = (
174             _DEFAULT_CONTAINER_OVERRIDES
175             if container_overrides is None
176             else tuple(container_overrides)
177         )
178
179     def run_infer(
180         self,
181         spec: ComputeJobSpec,
182         *,
183         on_wait: "Callable[[float], None] | None" = None,
184     ) -> ComputeResult:
185         out_root = spec.out_uri.rstrip("/")
186         token = self._token or uuid.uuid4().hex
187         done_uri = f"{out_root}/_dispatch/{token}.done"
188         argv: List[str] = [
189             "infer",
190             "--video-uri",
191             spec.video_uri,
192             "--out-uri",
193             spec.out_uri,
194             "--done-uri",
195             done_uri,
196         ]
197         if spec.segmenter:
198             argv += ["--segmenter", spec.segmenter]
199         for kv in (*spec.config_overrides, *self._container_overrides):
200             argv += ["--set", kv]
201
202         execution = self._client.run_job(
203             self._job_name, argv, region=self._region, project=self._project
204         )
205         LOG.info(
206             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
207             self._job_name,
208             execution,
209             done_uri,
210         )

```

```

211
212         try:
213             marker = poll_until(
214                 lambda: self._read_marker(done_uri),
215                 self._policy,
216                 label="cloud-run-infer",
217                 logger=LOG,
218                 # Live heartbeat (#482): each still-waiting tick reaches the caller so
219                 # job.progress moves during the whole GPU run instead of freezing.
220                 on_tick=on_wait,
221             )
222         except PolicyTimeout as timeout_exc:
223             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
224             video_id = str(marker.get("video_id", ""))
225             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
226             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
227             rc = int(marker.get("returncode", 1))
228             if not video_id:
229                 LOG.warning(
230                     "cloud-run: job=%s completion marker present but empty/unreadable (no "
231                     "video_id) ? treating as failure",
232                     self._job_name,
233                 )
234                 rc = rc or 1
235             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
236             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
237             LOG.info(
238                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
239             )
240             return ComputeResult(
241                 returncode=rc,
242                 outputs_uri=outputs_uri,
243                 video_id=video_id,
244                 report=report,
245                 execution=str(execution or ""),
246             )
247
248     def _handle_poll_timeout(
249         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
250     ) -> dict:
251         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
252
253         The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed
job's own
254         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
255         ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
256         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
257         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
258         execution name so the operator can verify the state in the GCP console. A cancel
failure
259         NEVER masks the original timeout."""
260         try:
261             late = self._read_marker(done_uri)
262             except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
263                 LOG.warning(
264                     "cloud-run: final post-timeout marker check failed ? treating as
absent",
265                     exc_info=True,

```

```

266         )
267         late = None
268     if late is not None:
269         LOG.warning(
270             "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
271             "? rescuing the completed result",
272             self._job_name,
273             execution,
274         )
275         return late
276     try:
277         self._client.cancel_execution(execution)
278     except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
279         cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
280         LOG.warning(
281             "cloud-run: best-effort cancel of execution=%s failed: %s",
282             execution,
283             cancel_exc,
284             exc_info=True,
285         )
286     else:
287         cancel_note = "cancel requested"
288         LOG.info("cloud-run: requested cancel of execution=%s", execution)
289         raise PolicyTimeout(
290             f"{timeout_exc} [cloud-run job={self._job_name} "
291             f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
292             "state in the GCP console]"
293         ) from timeout_exc
294
295     # --- storage-backed completion signal (bucket-poll) ---
296     def _read_marker(self, done_uri: str) -> Optional[dict]:
297         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
298         ref = parse_uri(done_uri)
299         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
300             return None
301         return (
302             self._read_json(done_uri) or {}
303         ) # empty dict still terminates the poll (present == done)
304
305     def _read_json(self, uri: str) -> dict:
306         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
307         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
308         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
309         try:
310             ref = parse_uri(uri)
311             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
312                 dst = os.path.join(td, ref.name or "obj.json")
313                 local = fetch(uri, dst, config=self._storage_config)
314                 with open(local, "r", encoding="utf-8") as f:
315                     data: Any = json.load(f)
316                 return data if isinstance(data, dict) else {}
317         except Exception:
318             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
319             return {}
320

```

```

1      """Job-list endpoint (the processing-queue view) as its own router ? kept OUT of the
backend.main
2      god-module per the P24 route ceiling (see tests/test_main_module_budget.py; pattern:
backend/api/uploads.py).
3
4      GET /api/jobs is tenant-scoped and status-filtered so a client can discover in-flight
work it did NOT
5      submit (a page reload, a new tab, another device) and show its status ? which also stops
a tester who
6      can't see their running job from re-submitting the same video into a second, PAID GPU
run (#448). The
7      per-submit dedup that makes that safe server-side lives inline in
backend.main.process_video.
8
9      Reads the live DB + config off ``request.app.state`` (set in backend.main.create_app),
so it needs no
10     module-global wiring and stays trivially testable via the standard
TestClient(create_app(cfg, db)).
11     """
12
13     import tempfile
14     from typing import Any, List, Optional, Tuple
15
16     from fastapi import APIRouter, HTTPException, Request
17     from fastapi.responses import JSONResponse
18
19     from backend.api import (
20         auth as edge_auth, # M9: Cloudflare Access edge-auth (default-OFF)
21     )
22     from backend.utils.freespace import (
23         require_free_bytes, # HTTP-507 scratch guard (P2, D11)
24     )
25
26     router = APIRouter()
27
28
29     def claim_process_job(
30         db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
31     ) -> Optional[Tuple[bool, Optional[JSONResponse]]]:
32         """ATOMIC idempotent-submit hook for backend.main.process_video (PR #481 review):
delegates to
33         ``db.claim_or_create_process_job`` (ONE write-locked op) so two concurrent identical
submits ? a
34         double-click / retry ? can't both create a job and start a second PAID GPU run.
Returns
35         ``(created, dedup_response)``: when ``created`` is True the caller dispatches; when
False the caller
36         just returns ``dedup_response`` (a 202 pointing at the already-active job). ``None``
on DB failure.
37         ``req.force`` opts out (deliberate reprocess). Keyed on (owner_id, video_path) ? the
input isn't
38         hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
39         the submit-time md5 when cheaply known (#484) ? persisted at CREATE so a restarted
control plane
40         can re-associate the job with its bucket outputs (CP-rebuild $4.2); "" ? the drain
fills it."""
41         claimed = db.claim_or_create_process_job(
42             job_id,
43             req.video_path,
44             segmenter=req.segmenter,
45             player_pool=req.player_pool,
46             max_frames=req.max_frames,
47             owner_id=owner_id,
48             locale=req.locale,

```

```

49         compute=req.compute,
50         force=req.force,
51         video_id=video_id,
52     )
53     if claimed is None:
54         return None
55     if claimed["created"]:
56         return (True, None)
57     return (
58         False,
59         JSONResponse(
60             status_code=202,
61             content={
62                 "job_id": claimed["job_id"],
63                 "status": claimed["status"],
64                 "poll": f"/api/jobs/{claimed['job_id']}",
65                 "deduplicated": True,
66             },
67         ),
68     )
69
70
71 def _fetch_scratch_guard(cfg: Any, input_ref: Any) -> None:
72     """P2/D11 507 fast-fail for a remote input the DRAIN will fetch to local scratch
73     (hash + validate). Raises ``HTTPException(507)`` when the box can't hold the
74     copy."""
75     from backend import orchestration # call-time seam (mirrors submit_process_job)
76     need = orchestration._remote_input_size_bytes(input_ref, getattr(cfg, "storage",
77     None))
78     require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
79
80 def submit_process_job(
81     db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
82 ) -> Optional[Tuple[bool, JSONResponse]]:
83     """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in
84     one hook.
85
86     Order matters (#492 review):
87     1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side
88     effects.
89     2. The remote-input scratch guard (507) runs ONLY on a probe miss: a cache hit never
90     fetches to local scratch, so low temp space must not reject it (review blocker
91     2).
92     3. CLAIM atomically (#481). A concurrent identical submit dedups here ? BEFORE any
93     re-hydrate side effect, so the first request owns the cache completion (blocker
94     1).
95     4. Only the claim WINNER realizes the hit (``cached_process_result`` ? itself
96     serialized
97     per video_id) and completes the job instantly: no drain, no GPU dispatch.
98
99     Returns ``(queued, response)``: ``queued`` True ? the caller kicks the drain;
100     ``None`` ?
101     DB failure (the caller 500s)."""
102     from backend import orchestration # call-time seam: tests patch orchestration attrs
103
104     video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
105     probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
106         db, cfg, video_id
107     )
108     if input_ref.backend != "local" and not probed:
109         _fetch_scratch_guard(cfg, input_ref) # raises 507 pre-claim, exactly as before
110     outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
111     if outcome is None:

```

```

106         return None
107     created, dedup_response = outcome
108     if not created:
109         assert dedup_response is not None # claim contract: not-created ? dedup
response
110         return (False, dedup_response)
111     if probed:
112         cached = orchestration.cached_process_result(
113             db, cfg, video_id, req.video_path, requested_compute=req.compute
114         )
115         if cached is not None:
116             db.mark_job_done(job_id, cached)
117             return (
118                 False,
119                 JSONResponse(
120                     status_code=202,
121                     content={
122                         "job_id": job_id,
123                         "status": "done",
124                         "poll": f"/api/jobs/{job_id}",
125                         "cached": True,
126                     },
127                 ),
128             )
129     # The probe said hit but realizing it failed (report vanished / corrupt csv):
fall
130     # back to the normal queued dispatch ? but the scratch guard was skipped above,
so it
131     # must run NOW. If it fails, the already-claimed row is marked failed BEFORE
raising
132     # (a bare raise would leave a phantom 'queued' job for the drain to grab).
133     if input_ref.backend != "local":
134         try:
135             _fetch_scratch_guard(cfg, input_ref)
136         except HTTPException:
137             db.mark_job_failed(
138                 job_id,
139                 "cache re-hydrate failed and local scratch cannot hold a fetch of "
140                 "the source; free disk space or retry later",
141             )
142             raise
143     return (
144         True,
145         JSONResponse(
146             status_code=202,
147             content={
148                 "job_id": job_id,
149                 "status": "queued",
150                 "poll": f"/api/jobs/{job_id}",

```

18. user (2026-07-06T17:11:15.525Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local:``/``
8     is an identity passthrough; ``s3:``/`` / ``gcs:``/`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?

```

```

11     the path validated end-to-end on DigitalOcean (`docs/SETUP_NEW_MACHINE.md`).
12
13     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14     (validator registry ? segmenter ? report), never a forked pipeline.
15     """
16
17     from __future__ import annotations
18
19     import argparse
20     import csv
21     import json
22     import logging
23     import os
24     import re
25     import sys
26     import tempfile
27     from typing import Any, List
28
29     from backend import storage
30     from backend.compute import ComputeJobSpec, get_compute_backend
31     from backend.config import (
32         StartupCheckError,
33         enforce_startup_checks,
34         load_config,
35         probe_cuda_available,
36     )
37     from backend.infrastructure.database import Database
38     from backend.pipeline.segmenters.base import segmenter_registry
39     from backend.pipeline.validators.base import registry as validator_registry
40     from backend.reporting import emit_indexer_report
41     from backend.results import Failure
42     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("backend.worker")
46
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
50         low = v.lower()
51         if low in ("true", "false"):
52             return low == "true"
53         for cast in (int, float):
54             try:
55                 return cast(v)
56             except ValueError:
57                 pass
58         return v
59
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63         for kv in sets or []:
64             key, sep, val = kv.partition("=")
65             if not sep:
66                 raise ValueError(f"--set expects k=v, got {kv!r}")
67             parts = key.split(".")
68             obj = config
69             for p in parts[:-1]:
70                 obj = getattr(obj, p)
71             setattr(obj, parts[-1], _coerce(val))
72

```

```

73
74 def _write_intervals_csv(segments: List[dict], path: str) -> None:
75     """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76     artifact (same schema as the rally-annotator labels)."""
77     with open(path, "w", newline="") as f:
78         w = csv.writer(f)
79         w.writerow(["start", "end", "shot_count", "ending_reason"])
80         for s in segments:
81             w.writerow(
82                 [
83                     f"{float(s.get('start_time', 0.0)):.3f}",
84                     f"{float(s.get('end_time', 0.0)):.3f}",
85                     s.get("shot_count", ""),
86                     s.get("ending_reason", s.get("shot_count_confidence", "")),
87                 ]
88             )
89
90
91 def _write_report_json(
92     video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93 ) -> None:
94     with open(path, "w") as f:
95         json.dump(
96             {
97                 "video_id": video_id,
98                 "status": status,
99                 "segment_count": len(segments),
100                 # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                 # control plane re-hydrating from outputs/<md5>/ restores the video row
102                 # with its truthful path ? without this, only the segments recover.
103                 "video_uri": video_uri,
104                 "segments": [
105                     {
106                         "start": float(s.get("start_time", 0.0)),
107                         "end": float(s.get("end_time", 0.0)),
108                     }
109                     for s in segments
110                 ],
111             },
112             f,
113             indent=2,
114         )
115
116
117 def _run_startup_checks(config: Any) -> bool:
118     """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
119     best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
120     cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
121     (returns True, ZERO work) when no deployment.profile is selected.
122
123     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
124     torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
125     WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
126     no-op of work, not just of output."""
127     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
128     cuda = (
129         probe_cuda_available() if profile else None
130     ) # torch-free on the default-OFF path

```

```

131         try:
132             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
133             return True
134         except StartupCheckError:
135             return False # already logged at ERROR by enforce_startup_checks
136
137
138     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
139         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
140         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
141         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
142
143         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
144         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
145         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
146         from backend.infrastructure.execution_policy import PolicyTimeout
147
148         spec = ComputeJobSpec(
149             video_uri=args.video_uri,
150             out_uri=args.out_uri,
151             segmenter=args.segmenter,
152             config_overrides=tuple(args.set or ()),
153         )
154         try:
155             result = compute.run_infer(spec)
156         except PolicyTimeout as e:
157             logger.warning(
158                 "[worker] remote compute did not complete (no marker within budget): %s", e
159             )
160             return 4
161         logger.info(
162             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
163             result.video_id,
164             result.returncode,
165             result.outputs_uri,
166         )
167         return result.returncode
168
169
170     def _write_done_marker(
171         done_uri: str,
172         video_id: str,
173         returncode: int,
174         segment_count: int,
175         status: str,
176         storage_cfg: dict,
177         scratch: str,
178     ) -> None:
179         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
180         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
181         try:
182             marker = {
183                 "video_id": video_id,
184                 "returncode": returncode,
185                 "segment_count": segment_count,
186                 "status": status,
187             }

```

```

188         local = os.path.join(scratch, "_done.json")
189         with open(local, "w", encoding="utf-8") as f:
190             json.dump(marker, f)
191         storage.put(local, done_uri, config=storage_cfg)
192         logger.info("[worker] wrote completion marker -> %s", done_uri)
193     except Exception as e: # never fail a good run because the marker push hiccuped
194         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
195
196
197     def cmd_infer(args: argparse.Namespace) -> int:
198         config = load_config(args.config) if args.config else load_config()
199         _apply_overrides(config, args.set)
200         if not _run_startup_checks(config):
201             return 2
202
203         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
204         # process.
205         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
206         # through to the
207         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
208         # (the
209         # dispatcher forces compute_target=local_gpu), so it never recursively re-
210         # dispatches.
211         compute = get_compute_backend(config)
212         if compute is not None:
213             return _dispatch_remote(compute, args)
214
215         storage_cfg = config.storage.model_dump()
216
217         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
218         os.makedirs(scratch, exist_ok=True)
219         config.output.output_dir = scratch
220
221         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
222         # to scratch.
223         local_video = storage.fetch(
224             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
225         )
226         print(f"[worker] pulled {args.video_uri} -> {local_video}")
227
228         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
229         # byte-identical).
230         video_id = compute_video_id(local_video)
231
232         # 3. VALIDATE (same registry as the main CLI).
233         val_results, val_context = validator_registry.run_all_with_context(
234             local_video, config
235         )
236         failures = [r for r in val_results if not r.passed]
237         db = Database(os.path.join(scratch, config.output.db_name))
238         db.add_video(
239             video_id,
240             local_video,
241             "failed" if failures else "processed",
242             [r.model_dump() for r in val_results],
243         )
244
245     def _fail(rc: int) -> int:
246         # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
247         # so a remote
248         # dispatcher's bucket-poll terminates FAST + accurately instead of hanging until
249         # its poll
250         # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
251         # local run).
252         if getattr(args, "done_uri", None):

```

```

244         _write_done_marker(
245             args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
246         )
247     return rc
248
249     segments: List[dict] = []
250     segmenter_actual = None
251     segmentation_ran = False
252     status = "failed"
253     try:
254         if failures:
255             print(
256                 "[worker] validation FAILED: "
257                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
258             )
259             return _fail(2)
260         seg_name = args.segmenter or config.indexing.default_segmenter
261         segmenter_cls = segmenter_registry.get(seg_name)
262         if not segmenter_cls:
263             print(
264                 f"[worker] unknown segmenter: {seg_name!r} "
265                 f"(have {list(segmenter_registry.list_available())})"
266             )
267             return _fail(2)
268         segmenter_actual = seg_name
269         result = segmenter_cls(db, config).process_video(
270             video_id, local_video, max_frames=args.max_frames
271         )
272         segmentation_ran = True
273         if isinstance(result, Failure):
274             print(f"[worker] segmenter FAILED: {result.message}")
275             return _fail(3)
276         segments = result.value if hasattr(result, "value") else result
277         status = "success"
278         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
279         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
280         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
281         if not segments:
282             detector_impl = (
283                 os.environ.get("RALLY_DETECTOR_IMPL")
284                 or getattr(config.indexing, "detector_impl", None)
285                 or "auto"
286             )
287             logger.warning(
288                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
289                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
290                 "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
291                 "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
292                 "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
293                 "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
294                 "detections, or the input resolution differs from the tuned proxy
resolution. "
295                 "Re-run with -v for the native command + frame counts.",
296                 video_id,
297                 segmenter_actual,
298                 detector_impl,
299             )

```

```

300     finally:
301         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
302         emit_indexer_report(
303             db=db,
304             video_id=video_id,
305             video_path=local_video,
306             config=config,
307             val_results=val_results,
308             val_context=val_context,
309             segmenter_requested=args.segmenter,
310             segmenter_actual=segmenter_actual,
311             was_reingest=False,
312             segmentation_ran=segmentation_ran,
313         )
314
315     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
316     out_local = os.path.join(scratch, "outputs", video_id)
317     os.makedirs(out_local, exist_ok=True)
318     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
319     _write_report_json(
320         video_id,
321         segments,
322         status,
323         os.path.join(out_local, "report.json"),
324         video_uri=str(getattr(args, "video_uri", "") or ""),
325     )
326
327     out_prefix = args.out_uri.rstrip("/")
328     pushed = []
329     for fn in sorted(os.listdir(out_local)):
330         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
331         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
332         pushed.append(uri)
333
334     print(
335         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}"
artifact(s):"
336     )
337     for u in pushed:
338         print("    ", u)
339
340     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
341     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
342     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
343     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
344     if getattr(args, "done_uri", None):
345         _write_done_marker(
346             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
347         )
348     return 0
349
350
351     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
352     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
353     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
354
355     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,

```

```

356     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
357     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
358     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
359
360
361     def _label_clip_name(fname: str) -> str:
362         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
363
364         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
365         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
366
367         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
368         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
369         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
370         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
371
372         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
373         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
374         """
375         name = fname.replace(".rallies.csv", "").replace(".csv", "")
376         return _LABEL_DATE_PREFIX.sub("", name)
377
378
379     def _probe_video_fps_width(path: str):
380         """Robust ffmpeg (fps, frame_width) for a real video. Returns None on ANY failure.
381
382         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
383         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
384         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
385         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
386         probe with ffmpeg (same tool as ``get_video_duration``) and return None so the
caller SKIPS
387         rather than guesses.
388         """
389         import json
390         import subprocess
391
392         try:
393             out = subprocess.check_output(
394                 [
395                     ffmpeg_bin(),
396                     "-v",
397                     "error",
398                     "-select_streams",
399                     "v:0",
400                     "-show_entries",
401                     "stream=r_frame_rate,width",
402                     "-of",
403                     "json",
404                     path,
405                 ],
406                 stderr=subprocess.DEVNULL,

```

```

407         ).decode()
408         st = (json.loads(out).get("streams") or [{}])[0]
409         num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
410         fps = float(num) / float(den or "1")
411         width = float(st.get("width") or 0)
412         if fps > 0 and width > 0:
413             return fps, width
414     except (
415         subprocess.SubprocessError,
416         ValueError,
417         KeyError,
418         OSError,
419         json.JSONDecodeError,
420     ):
421         pass
422     return None
423
424
425 def _resolve_train_detector(args: argparse.Namespace, config: Any):
426     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
427     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
428     box, stub=CPU mock). Returns the runner, or prints an error + returns None.
429
430     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
431     the detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
432     has no shuttle detector and is rejected.
433     """
434     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
435
436     seg_cls = segmenter_registry.get(args.segmenter)
437     if not seg_cls:
438         print(
439             f"[worker] unknown segmenter: {args.segmenter!r} "
440             f"(have {list(segmenter_registry.list_available())})"
441         )
442         return None
443     seg = seg_cls(None, config)
444     if not isinstance(seg, TrajectoryHybridSegmenter):
445         print(
446             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
447             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
448         )
449         return None
450     try:
451         runner, _ = seg._resolve_runner(config.indexing)
452     except NotImplementedError as e:
453         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
454         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
455         traceback.print_exc()
456         print(f"[worker] detector not available: {e}")
457         return None
458     ok, msg = runner.healthcheck()
459     if not ok:
460         print(f"[worker] detector environment not ready: {msg}")
461         return None
462     print(f"[worker] detector ready ({args.segmenter}): {msg}")
463     return runner
464
465 def cmd_train(args: argparse.Namespace) -> int:
466     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
467     shell out

```

```

467         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
468
469     Two trajectory sources (the train pipeline + harness are identical either way):
470     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
471         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
472     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
473         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
474         is the M3.5 "first real training" path; only the trajectory source flips.
475     """
476     import subprocess
477
478     config = load_config(args.config) if args.config else load_config()
479     _apply_overrides(config, args.set)
480     if not _run_startup_checks(config):
481         return 2
482     storage_cfg = config.storage.model_dump()
483
484     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
485     labels_dir = os.path.join(scratch, "labels")
486     traj_dir = os.path.join(scratch, "trajectories")
487     videos_dir = os.path.join(scratch, "videos")
488     os.makedirs(labels_dir, exist_ok=True)
489     os.makedirs(traj_dir, exist_ok=True)
490
491     corpus = args.corpus_uri.rstrip("/")
492     label_uris = sorted(
493         u
494         for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
495         if u.lower().endswith(".csv")
496     )
497     if len(label_uris) < 2:
498         print(
499             f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
500             f"under {corpus}/labels/"
501         )
502         return 2
503
504     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
505     runner = None
506     video_by_stem: dict = {}
507     if args.trajectory_source == "detector":
508         os.makedirs(videos_dir, exist_ok=True)
509         runner = _resolve_train_detector(args, config)
510         if runner is None:
511             return 2
512         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
513             vname = storage.parse_uri(vuri).name
514             if not vname.lower().endswith(_VIDEO_EXTS):
515                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
516             video_by_stem[os.path.splitext(vname)[0]] = vuri
517
518     print(f"[worker] trajectory source: {args.trajectory_source}")
519     specs: List[dict] = []
520     for uri in label_uris:
521         fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
522         name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
523         local_label = storage.fetch(
524             uri, os.path.join(labels_dir, fname), config=storage_cfg

```

```

525         )
526     if args.trajectory_source == "detector":
527         vuri = video_by_stem.get(name)
528         if not vuri:
529             print(
530                 f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
531             )
532             continue
533         local_video = storage.fetch(
534             vuri,
535             os.path.join(videos_dir, storage.parse_uri(vuri).name),
536             config=storage_cfg,
537         )
538         video_id = compute_video_id(local_video)
539         # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
540         # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
541         meta = _probe_video_fps_width(local_video)
542         if meta is None:
543             print(
544                 f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
545             )
546             continue
547         fps, frame_width = meta
548         traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
549         if not traj:
550             print(
551                 f"[worker] detector produced no trajectory for {name!r} ? skipping."
552             )
553             continue
554         specs.append(
555             {
556                 "name": name,
557                 "trajectory": traj,
558                 "labels": local_label,
559                 "fps": fps,
560                 "frame_width": frame_width,
561             }
562         )
563     else:
564         from backend.pipeline.detectors.stub_runner import (
565             synth_trajectory_from_labels,
566         )
567
568         traj = os.path.join(traj_dir, f"{name}_traj.csv")
569         synth_trajectory_from_labels(
570             local_label, traj, fps=args.fps, frame_width=args.frame_width
571         )
572         specs.append(
573             {
574                 "name": name,
575                 "trajectory": traj,
576                 "labels": local_label,
577                 "fps": args.fps,
578                 "frame_width": args.frame_width,
579             }
580         )
581
582     if len(specs) < 2:
583         print(
584             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
585         )

```

```

586         return 2
587     manifest_path = os.path.join(scratch, "manifest.json")
588     with open(manifest_path, "w", encoding="utf-8") as f:
589         json.dump(specs, f, indent=2)
590     src_label = (
591         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
592     )
593     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
594
595     out_dir = os.path.join(scratch, "artifacts")
596     cmd = [
597         sys.executable,
598         "-m",
599         "training.gen0.harness",
600         "--manifest",
601         manifest_path,
602         "--out",
603         out_dir,
604         "--version",
605         args.version,
606     ]
607     if args.floor:
608         floor_local = (
609             storage.fetch(
610                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
611             )
612             if "://" in args.floor
613             else args.floor
614         )
615         cmd += ["--floor", floor_local]
616     print("[worker] training:", " ".join(cmd))
617     rc = subprocess.run(cmd).returncode
618     if rc != 0:
619         print(f"[worker] gen0 harness failed (rc={rc})")
620         return rc
621
622     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
623     bundle = os.path.join(out_dir, args.version)
624     out_prefix = args.out_uri.rstrip("/")
625     pushed = []
626     for fn in sorted(os.listdir(bundle)):
627         uri = f"{out_prefix}/weights/{args.version}/{fn}"
628         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
629         pushed.append(uri)
630     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
631     for u in pushed:
632         print("    ", u)
633     return 0
634
635
636 def build_parser() -> argparse.ArgumentParser:
637     p = argparse.ArgumentParser(
638         prog="python -m backend.worker",
639         description="Storage-aware worker: pull ? run pipeline ? push.",
640     )
641     p.add_argument(
642         "-v",
643         "--verbose",
644         action="store_true",
645         help="DEBUG logging (the native detector command, frame counts, per-stage
646         detail)",
647     )
648     sub = p.add_subparsers(dest="cmd", required=True)
649     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")

```

```

650     pi.add_argument(
651         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
652     )
653     pi.add_argument(
654         "--out-uri",
655         required=True,
656         help="output prefix (outputs/<video_id>/? is appended)",
657     )
658     pi.add_argument(
659         "--segmenter", default=None, help="default: indexing.default_segmenter"
660     )
661     pi.add_argument(
662         "--config", default=None, help="config.json path (default: ./config.json)"
663     )
664     pi.add_argument(
665         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
666     )
667     pi.add_argument("--max-frames", type=int, default=None)
668     pi.add_argument(
669         "--done-uri",
670         default=None,
671         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
672         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
673         "Default-OFF: omit it for the normal local run.",
674     )
675     pi.add_argument(
676         "--set",
677         action="append",
678         default=[],
679         metavar="k=v",
680         help="config override, e.g. --set indexing.skip_ai_handoff=true",
681     )
682     pi.set_defaults(func=cmd_infer)
683
684     pt = sub.add_parser(
685         "train",
686         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
687     )
688     pt.add_argument(
689         "--corpus-uri",
690         required=True,
691         help="prefix containing labels/ (+ videos/ for "
692         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
693     )
694     pt.add_argument(
695         "--out-uri",
696         required=True,
697         help="output prefix (weights/<version>/ is appended)",
698     )
699     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
700     pt.add_argument(
701         "--trajectory-source",
702         choices=["synth", "detector"],
703         default="synth",
704         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
705         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
706         "(needs videos/ in the corpus; honours detector_impl).",
707     )
708     pt.add_argument(
709         "--segmenter",
710         default="wasb_hybrid",
711         help="trajectory hybrid whose detector is used for --trajectory-source
712         detector",
713     )
714     pt.add_argument(

```

```

714         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
715     )
716     pt.add_argument("--config", default=None)
717     pt.add_argument("--scratch", default=None)
718     pt.add_argument(
719         "--fps",
720         type=float,
721         default=30.0,
722         help="synth trajectory fps (detector probes the video)",
723     )
724     pt.add_argument(
725         "--frame-width",
726         type=float,
727         default=1280.0,
728         help="synth trajectory frame width (detector probes the video)",
729     )
730     pt.add_argument("--set", action="append", default=[], metavar="k=v")
731     pt.set_defaults(func=cmd_train)
732     return p
733
734
735     def _setup_logging(verbose: bool) -> None:
736         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
737         diagnostics are
738         VISIBLE ? the worker previously ran with no logging config, so every
739         segmenter/detector log
740         (detector choice, 'Running native WASB inference', availability failures) was
741         silently dropped.
742         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
743         (the CLI
744         entry), not at import, so importing the module never reconfigures a caller's
745         logging."""
746         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
747         library
748         # configured logging at import), which would leave the level at WARNING and re-
749         suppress the
750         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
751         verbosity.
752         logging.basicConfig(
753             level=logging.DEBUG if verbose else logging.INFO,
754             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
755             datefmt="%H:%M:%S",
756             force=True,
757         )
758
759     def main(argv=None) -> int:
760         args = build_parser().parse_args(argv)
761         _setup_logging(getattr(args, "verbose", False))
762         return args.func(args)
763
764     if __name__ == "__main__":
765         sys.exit(main())
766

```

19. assistant (2026-07-06T17:11:18.357Z)

Now let me search for how the control plane enqueues work and check for the rally-control-plane service integration:

20. user (2026-07-06T17:11:19.108Z)

```

1     """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2

```

```

3      COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4      (``backend/storage/``): a ``ComputeBackend`` decides where the pure
DETECT?WINDOW?STITCH
5      core executes ? in-process on the local box (truly-local) or on a Cloud Run GPU Job
6      (cloud-serving) ? without the core ever branching on the profile (D1).
7
8      The seam is default-OFF: only ``deployment.compute_target == "cloud_run"`` selects a
9      non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10     existing in-process path byte-identical ? the factory returns ``None`` (= "run
inline").
11     ""
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Callable, Optional
18
19
20     @dataclass(frozen=True)
21     class ComputeJobSpec:
22         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24         deterministic regardless of where the job ran."""
25
26         video_uri: str # input (gs://? for cloud; a path/local:// for local)
27         out_uri: str # output ROOT (gs://? bucket for cloud)
28         segmenter: Optional[str] = None
29         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30         config_overrides: tuple[str, ...] = ()
31
32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus 4 = remote dispatch
timeout or an
37         empty/unreadable completion marker (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44         # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
45         # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
46         # executions describe <execution>). Empty for the in-process path / backends without
one.
47         execution: str = ""
48
49
50     class ComputeBackend(ABC):
51         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
52         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud

```

```

53         execution identically."""
54
55     @abstractmethod
56     def run_infer(
57         self,
58         spec: ComputeJobSpec,
59         *,
60         on_wait: "Callable[[float], None] | None" = None,
61     ) -> ComputeResult:
62         """Run ``spec`` to completion and return its ``ComputeResult``.
63
64         ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
65         still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
66         so a
67         long GPU run doesn't look frozen. Implementations without a wait loop may ignore
68         it."""

```

21. user (2026-07-06T17:11:20.373Z)

Found 10 files

- backend\worker__main__.py
- backend\orchestration.py
- backend\infrastructure\database_jobs.py
- backend\api\job_worker.py
- backend\compute\cloud_run.py
- backend\compute\base.py
- backend\config\models.py
- backend\compute__init__.py
- backend\config\presets.py
- backend\infrastructure\database.py

22. user (2026-07-06T17:11:21.214Z)

```

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/backend/compute/__init__.py:79:    job_name = os.environ.get("CLOUD_RUN_JOB", "rally-
worker")
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/backend/orchestration.py:608:
"job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/backend/worker/__main__.py:213:    scratch = args.scratch or
tempfile.mkdtemp(prefix="rally-worker-")
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/deploy/cloudrun/gen_service_yaml.py:29:    with it; the code default ("rally-worker") is
already correct.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/deploy/cloudrun/gen_service_yaml.py:156:    help="Container image; default = the
project's rally-worker:latest (post-#470 it "
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/deploy/cloudrun/gen_service_yaml.py:193:    args.image =
f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-worker:latest"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/tests/test_api_cloud_dispatch.py:486:    monkeypatch.setenv("CLOUD_RUN_JOB", "rally-
worker")
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/tests/test_api_cloud_dispatch.py:661:    monkeypatch.setenv("CLOUD_RUN_JOB", "rally-
worker")
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/tests/test_compute_backend.py:94:    EXEC_NAME = "projects/p/locations/asia-
south1/jobs/rally-worker/executions/exec-slow"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/tests/test_compute_backend.py:151:    "rally-worker", ["infer"],
region="asia-south1", project=missing
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer/tests/test_compute_backend.py:175:    job_name="rally-worker",

```

```

C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer/tests/test_compute_backend.py:196:         client.calls[0]["job"] == "rally-worker"
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer/tests/test_compute_backend.py:328:         job_name="rally-worker",
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer/tests/test_compute_backend.py:350:         job_name="rally-worker",
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer/tests/test_compute_backend.py:762:         job_name="rally-worker",
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer/tests/test_drain_concurrency.py:46:     monkeypatch.setenv("CLOUD_RUN_JOB", "rally-
worker")
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer/tests/test_restart_recovery.py:431:     monkeypatch.setenv("CLOUD_RUN_JOB", "rally-
worker")
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer/tests/test_restart_recovery.py:496:     monkeypatch.setenv("CLOUD_RUN_JOB", "rally-
worker")

```

23. assistant (2026-07-06T17:11:22.897Z)

Let me check the compute/___init___py to see how the backend is initialized:

24. user (2026-07-06T17:11:23.138Z)

```

1      """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3      COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4      ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5      ``None``
6      otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
7      identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
8      ``cpu_mock`` nothing
9      changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
10     """
11
12     from __future__ import annotations
13
14     import dataclasses
15     import os
16     from typing import Any, Optional
17
18     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
19     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
20
21     __all__ = [
22         "ComputeBackend",
23         "ComputeJobSpec",
24         "ComputeResult",
25         "get_compute_backend",
26     ]
27
28     def _storage_dict(config: Any) -> dict:
29         storage = getattr(config, "storage", None)
30         if storage is None:
31             return {}
32         if hasattr(storage, "model_dump"):
33             return storage.model_dump() # type: ignore[no-any-return]
34         return dict(storage) if isinstance(storage, dict) else {}
35
36     def get_compute_backend(
37         config: Any,

```

```

38     *,
39     run_client: Optional[Any] = None,
40     policy: ExecutionPolicy = DEFAULT_POLICY,
41     token: Optional[str] = None,
42     target_override: Optional[str] = None,
43 ) -> Optional[ComputeBackend]:
44     """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
45     in-process (the default-OFF path).
46
47     Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
48     Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
49     / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
50     pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
51
52     ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
53     per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
54     server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
55     """
56     target = (
57         target_override
58         if target_override is not None
59         else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
60     )
61     if target != "cloud_run":
62         return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
63
64     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
65     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
66
67     # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
68     # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
69     # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the
old
70     # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
71     # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
72     if policy is DEFAULT_POLICY:
73         max_wait = getattr(
74             getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
75         )
76         if max_wait:
77             policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
78
79     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
80     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
81     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
82     client = run_client if run_client is not None else GoogleCloudRunJobClient()
83     return CloudRunCompute(
84         run_client=client,
85         job_name=job_name,
86         region=region,
87         project=project,
88         storage_config=_storage_dict(config),
89         policy=policy,
90         token=token,

```

91)
92

25. user (2026-07-06T17:11:23.362Z)

```
1      """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3      The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4      Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5      and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6      Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8      SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9      monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11     the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote
31
32     from backend import storage
33     from backend.config import AppConfig, StitchingConfig
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36     from backend.pipeline.validators.base import registry
37     from backend.results import Failure
38     from backend.utils.paths import output_base, safe_output_filename
39
40     if (
41         TYPE_CHECKING
42     ): # runtime import would be circular (main imports us for the re-export)
43         from backend.main import ProcessRequest # noqa: F401
44
45     logger = logging.getLogger(__name__)
46
47     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
```

```

48     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
50     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
51     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
61         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
62         """
63         if segments:
64             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
65         else:
66             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
67
68
69     def _validation_failure_message(failures: List[Any]) -> str:
70         details = []
71         for failure in failures:
72             name = getattr(failure, "validator_name", None) or "validation"
73             message = (getattr(failure, "message", None) or "").strip()
74             details.append(f"{name}: {message}" if message else str(name))
75         if not details:
76             return "Video failed validation checks."
77         return "Video failed validation: " + "; ".join(details)
78
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
scratch."""
82         storage_config = (
83             config.get("storage")
84             if isinstance(config, dict)
85             else getattr(config, "storage", None)
86         )
87         try:
88             if storage.input_is_local(raw_input, storage_config):
89                 return config
90         except ValueError:
91             return config
92         try:
93             cfg = (
94                 config.model_dump(mode="json")
95                 if hasattr(config, "model_dump")
96                 else dict(config or {})
97             )
98         except Exception:
99             cfg = {}
100         report = dict(cfg.get("report") or {})
101         if report.get("output_dir"):
102             return config
103         report["output_dir"] = os.path.join(output_base(config), "reports")
104         cfg["report"] = report
105         return cfg

```

```

106
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
112         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
113         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
114         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
115         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
116         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
117         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
118         and prunes any partial rows (destroy-AFTER-confirm)."""
119         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
120         ref = storage.parse_uri(uri)
121         n = 0
122         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
123             local = storage.fetch(
124                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
125             )
126             with open(local, newline="", encoding="utf-8") as f:
127                 for row in csv.DictReader(f):
128                     if n >= max_rows:
129                         logger.warning(
130                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
131                             max_rows,
132                             uri,
133                         )
134                         break
135                     try:
136                         start, end = float(row["start"]), float(row["end"])
137                     except (KeyError, TypeError, ValueError):
138                         continue
139                     if not (math.isfinite(start) and math.isfinite(end)):
140                         continue # reject inf/nan times rather than persist a junk rally
141                     meta: Dict[str, Any] = {
142                         "source": "cloud_run",
143                         "ending_reason": (row.get("ending_reason") or "").strip(),
144                     }
145                     sc = (row.get("shot_count") or "").strip()
146                     if sc:
147                         try:
148                             meta["shot_count"] = int(
149                                 float(sc)
150                             ) # OverflowError on 'inf'/'1e400'
151                         except (ValueError, OverflowError):
152                             meta["shot_count"] = sc
153                     db.add_play_segment(video_id, start, end, metadata=meta)
154                     n += 1
155         return n
156
157
158     def _cloud_available(cfg) -> bool:
159         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
160
161         True when the control plane is wired for Cloud Run: either the configured default

```

```

target is
162         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
163         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
164         (``storage.output_root``)
165         are present so a forced per-request override can actually dispatch + collect a
result.
166         Surfed in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work. """
167         deployment = getattr(cfg, "deployment", None)
168         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
169             return True
170         has_job = bool(
171             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
172         )
173         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
174         return bool(has_job and out_root)
175
176
177     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
178         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
179         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
180         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
181         (queue/cold-start), so the very first tick already shows movement. """
182         minutes = int(elapsed_sec) // 60
183         if minutes < 1:
184             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
185         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
186
187
188     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
189         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
190         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
191         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
192         ``md5Hash``. Never raises ? a metadata miss must not block a submit. """
193         if getattr(input_ref, "backend", "") != "gcs":
194             return None
195         sc = _storage_settings(cfg)
196         if not sc:
197             return None
198         try:
199             st = storage.get_storage("gcs", config=sc).stat(input_ref)
200             return getattr(st, "md5", None) or None
201         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
202             logger.info(
203                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
204                 getattr(input_ref, "uri", input_ref),
205                 exc_info=True,
206             )
207         return None
208
209
210     def _cached_process_payload(
211         video_id: str, validation_results, segments, requested_compute
212     ) -> Dict[str, Any]:
213         """The /api/process result for a video whose detection already exists ? ONE shape
for
214         both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484). """

```

```

215         return {
216             "video_id": video_id,
217             "status": "success",
218             "message": (
219                 f"Reused {len(segments)} cached play segments. "
220                 "Pass force=true to reprocess this video."
221             ),
222             "results": validation_results or [],
223             "play_segments": segments,
224             "cached": True,
225             "compute": {"path": "cached", "requested": requested_compute},
226         }
227
228
229     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
230         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
231         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
232         decide."""
233         ref = storage.parse_uri(uri)
234         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
235             local = storage.fetch(
236                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
237             )
238             with open(local, encoding="utf-8") as f:
239                 loaded = json.load(f)
240             return loaded if isinstance(loaded, dict) else None
241
242     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
243     hits
244     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
245     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
246     instance
247     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
248     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
249     _REHYDRATE_LOCKS_GUARD = threading.Lock()
250
251     def _rehydrate_lock(video_id: str) -> threading.Lock:

```

26. assistant (2026-07-06T17:11:25.341Z)

Now let me read the critical part about how a job is dispatched (the orchestration run_infer call):

27. user (2026-07-06T17:11:26.201Z)

```

250     def _rehydrate_lock(video_id: str) -> threading.Lock:
251         with _REHYDRATE_LOCKS_GUARD:
252             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
253
254
255     def probe_process_cache(db, cfg, video_id: str) -> bool:
256         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
257         bucket)?
258
259         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
260         1
261         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
262         (blocker 2): a True probe means the request should never need CP scratch. Best-
263         effort:
264         errors report False (? the normal dispatch path with its guards)."""
265         try:
266             existing = db.get_video(video_id)
267             if (

```

```

265         existing
266         and existing.get("status") == "processed"
267         and db.query_play_segments(video_id, {}))
268     ):
269         return True
270     sc = _storage_settings(cfg)
271     if not sc or (sc.get("backend") or "local") == "local":
272         return False
273     output_root = (sc.get("output_root") or "").rstrip("/")
274     if not output_root:
275         return False
276     report_uri = f"{output_root}/outputs/{video_id}/report.json"
277     if not storage.exists(report_uri, config=sc):
278         return False
279     report = _read_bucket_json(report_uri, sc)
280     return bool(report and report.get("status") == "success")
281 except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
282     logger.warning(
283         "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
284     )
285     return False
286
287
288 def cached_process_result(
289     db, cfg, video_id: str, video_path: str, requested_compute=None
290 ) -> Optional[Dict[str, Any]]:
291     """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
292     the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
293     re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
294     source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
295     completed work found (or the check failed) ? the caller dispatches normally. The
bucket
296     branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
297     partial earlier ingest is replaced, never double-counted.
298
299     Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
300     per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
301     in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
302     job before invoking (writes happen post-claim)."""
303     try:
304         with _rehydrate_lock(video_id):
305             existing = db.get_video(video_id)
306             if existing and existing.get("status") == "processed":
307                 segs = db.query_play_segments(video_id, {})
308                 if segs:
309                     return _cached_process_payload(
310                         video_id,
311                         existing.get("validation_results", []),
312                         segs,
313                         requested_compute,
314                     )
315             sc = _storage_settings(cfg)
316             if not sc or (sc.get("backend") or "local") == "local":
317                 return None
318             output_root = (sc.get("output_root") or "").rstrip("/")
319             if not output_root:
320                 return None
321             outputs_uri = f"{output_root}/outputs/{video_id}"
322             report_uri = f"{outputs_uri}/report.json"

```

```

323         if not storage.exists(report_uri, config=sc):
324             return None
325         report = _read_bucket_json(report_uri, sc)
326         if not report or report.get("status") != "success":
327             return None
328         if not video_path:
329             # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
330             # carries the worker's --video-uri (P3); older reports fall back to the
331             # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
332             # recover, and the next same-md5 submit re-binds the truthful path via
333             # add_video's upsert.
334             video_path = str(report.get("video_uri") or "") or _first_object_under(
335                 f"{output_root}/videos/{video_id}/", sc
336             )
337             play_wm, cand_wm = db.segment_watermarks(video_id)
338             db.add_video(video_id, video_path, "processed", [])
339             n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
340             if n <= 0:
341                 return None
342             segments = [
343                 s
344                 for s in db.query_play_segments(video_id, {})
345                 if int(s.get("id", 0)) > play_wm
346             ]
347             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
348             if not segments:
349                 return None
350             logger.info(
351                 "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
352                 n,
353                 video_id,
354                 outputs_uri,
355             )
356             return _cached_process_payload(video_id, [], segments, requested_compute)
357     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
358         logger.warning(
359             "[API] submit-time cache check failed for %s ? dispatching normally",
360             video_id,
361             exc_info=True,
362         )
363         return None
364
365
366     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
367         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
368         try:
369             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
370         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
371             return ""
372
373
374     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
375
376
377     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
378         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
379         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
so
380         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
381         has the video's rows now (re-query). Non-md5 ids return False without touching the
382         bucket; every failure degrades to False (the read just reports what the DB has)."""

```

```

383     vid = (video_id or "").strip().lower()
384     if not _MD5_HEX_RE.fullmatch(vid):
385         return False
386     if not probe_process_cache(db, cfg, vid):
387         return False
388     return cached_process_result(db, cfg, vid, "") is not None
389
390
391     def _maybe_dispatch_remote(
392         cfg,
393         db,
394         req: "ProcessRequest",
395         video_id: str,
396         seg_name: str,
397         val_results,
398         play_wm: int,
399         cand_wm: int,
400         notify,
401     ) -> Optional[Tuple[Dict[str, Any], bool]]:
402         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
403         ``deployment.compute_target
404         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
405         intervals into
406         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
407
408         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
409         the cloud job
410         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
411         observability report
412         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
413         process segmenter
414         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
415         non-``cloud_run`` target, so
416         the in-process path stays byte-identical)."""
417         from backend.compute import ComputeJobSpec, get_compute_backend
418
419         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
420             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
421             (id > play_wm);
422             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
423             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
424             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
425             only when the
426             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
427             rc / ingest
428             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
429             records the
430             # intended backend, `requested` the picker choice, `dispatched` whether the
431             remote job ran.
432             return (
433                 {
434                     "video_id": video_id,
435                     "status": "failed",
436                     "message": msg,
437                     "results": [r.model_dump() for r in val_results],
438                     "play_segments": [],
439                     "compute": {
440                         "path": "cloud_run" if ran else "not_run",
441                         "requested": req.compute,
442                         "target": "cloud_run",
443                         "dispatched": ran,
444                     },
445                 },
446                 ran,
447             )

```

```

437
438     # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
439     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
440     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
441     choice = (req.compute or "").strip().lower()
442     if choice and choice not in ("local", "cloud"):
443         logger.warning(
444             "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
445             req.compute,
446         )
447     choice = ""
448     if choice == "local":
449         return None # explicit "run on my machine" ? in-process regardless of the
server's target
450     if choice == "cloud":
451         if not _cloud_available(cfg):
452             return _failed(
453                 "cloud-serving was requested but this server isn't configured for it "
454                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
455                 False,
456             )
457     compute = get_compute_backend(cfg, target_override="cloud_run")
458     else:
459         compute = get_compute_backend(
460             cfg
461         ) # server default (default-OFF unless cloud_run)
462     if compute is None:
463         return None # default-OFF ? run the in-process segmenter below (unchanged)
464
465     storage_cfg = getattr(cfg, "storage", None)
466
467     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
468     try:
469         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
470     except ValueError as e:
471         return _failed(
472             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
473         )
474     if input_ref.backend == "local":
475         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
476         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
477         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
478         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
479         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
480         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
481         # default, while a bare/local-looking root can't stage no matter what
input_backend says
482         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
483         # gives us nothing to build the staged URI under).
484         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
485         try:
486             root_backend = (

```

```

487         storage.parse_uri(input_root).backend if input_root else "local"
488     )
489     except ValueError: # unsupported scheme in input_root ? nowhere real to stage
490         to
491         root_backend = "local"
492         if root_backend == "local":
493             return _failed(
494                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
495                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
496                 "Run job.",
497                 False,
498             )
499         local_src = input_ref.key # resolved local path of the upload
500         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
501         try:
502             notify("staging upload to bucket")
503             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
504             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
505             storage.put(
506                 local_src,
507                 staged_uri,
508                 config=(
509                     storage_cfg.model_dump() if storage_cfg is not None else None
510                 ),
511             )
512             except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
513             logger.error(
514                 "[API] cloud-serving: failed to stage %s -> %s: %s",
515                 local_src,
516                 staged_uri,
517                 e,
518                 exc_info=True,
519             )
520             return _failed(
521                 f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
522                 False,
523             )
524             # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
525             # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
526             # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
527             req.video_path = staged_uri
528             input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
529             db.add_video(
530                 video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
531             )
532             out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
533             if not out_root:
534                 return _failed(
535                     "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
536                     "? that's where the Cloud Run job writes the result.",
537                     False,
538                 )
539             notify("dispatching (cloud_run)")

```

```

540     logger.info(
541         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
542         req.video_path,
543         out_root,
544     )
545     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
546     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
547     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
548     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
549     overrides = []
550     sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
551     if sk is not None:
552         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
553     spec = ComputeJobSpec(
554         video_uri=req.video_path,
555         out_uri=out_root,
556         segmenter=seg_name,
557         config_overrides=tuple(overrides),
558     )
559     try:
560         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
561         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
562         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
563         # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
564         result = compute.run_infer(
565             spec,
566             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
567         )
568     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
569         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
570         return _failed(f"cloud dispatch error: {e}", True)
571     if result.returncode != 0:
572         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
573     if result.video_id and result.video_id != video_id:
574         # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
575         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
576         logger.warning(
577             "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
578             "and the worker hash; ingesting under the local id.",
579             result.video_id,
580             video_id,
581         )
582
583     notify("ingesting (cloud_run)")
584     storage_dict = (
585         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
586         if hasattr(storage_cfg, "model_dump")
587         else {}
588     )
589     try:
590         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
591     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
592         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)

```

```

593         return _failed(
594             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
595             True,
596         )
597
598     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
599     segments = [
600         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
601     ]
602     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
603     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
604     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
605     provenance = {
606         "path": "cloud_run",
607         "requested": req.compute,
608         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
609         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
610         "execution": result.execution,
611         "outputs_uri": result.outputs_uri,
612     }
613     logger.info(
614         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
615         result.execution,
616         provenance["job"],
617         provenance["region"],
618         result.outputs_uri,
619         video_id,
620     )
621     return (
622         {
623             "video_id": video_id,
624             "status": "success",
625             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
626             "results": [r.model_dump() for r in val_results],
627             "play_segments": segments,
628             "compute": provenance,
629         },
630         True,
631     )
632
633
634     def _execute_processing(
635         config=None, db=None, req=None, progress=None
636     ) -> Dict[str, Any]:
637         """The full hash ? validate ? segment ? report pipeline for one video.
638
639         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
640         module-level ``global_config``/``db_instance`` (backward-compatible with existing
641         tests that pass only ``req``). Route handlers always pass explicit params; older
642         test code that calls ``_execute_processing(req)`` still works.
643
644         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
645         Raises on unexpected internal errors ? the caller records those as a job failure
646         """
647         import backend.main as _main # call-time seam: main's globals/model stay patchable
648
649         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),

```

```

4:Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` / `_ingest_remote_segments`),
391:def _maybe_dispatch_remote(
564:    result = compute.run_infer(
703:    # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
780:    remote = _maybe_dispatch_remote(

```

29. assistant (2026-07-06T17:11:29.239Z)

Perfect! Now let me check the README.md in deploy/cloudrun/ to see what instructions are already there:

30. user (2026-07-06T17:11:30.258Z)

```

1      # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3      The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4      dispatches a Cloud Run Job execution running this image's worker on an L4, then
bucket-polls
5      a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7      > Status: the dispatch shim + image are landed + mock-tested (M6). The real
build + push +
8      > run on an L4 is the M7 owner step, within the $100/?10k budget (D17) ?
confirm cmd + hourly
9      > cost first, delete every resource after ([[gcp-vm-always-delete]]). CI does
not build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14     `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REPO=rally
20     # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21     REGION=asia-southeast1
22     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
23     # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
24     gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
25     ```
26     *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
D17.)*
27
28     ## 2. Create the Cloud Run Job (GPU, scale-to-zero)
29     The proven shape (first real M7 deploy, 2026-07-03): weights staged in a same-region
bucket
30     mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step
needed).
31     ```bash
32     gcloud run jobs create rally-worker \
33         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
34         --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
35         --cpu 8 --memory 32Gi \
36         --max-retries 0 --task-timeout 3600 \
37         --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
38         --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
39         --add-volume-mount volume=serving,mount-path=/gcs

```

```

40     ``
41     - **--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it
also halves
42     the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive
Y/n which,
43     in a non-interactive shell, **hangs forever with zero output and nothing created**. An
empty
44     `gcloud` that "did nothing" ? suspect a hidden prompt.
45     - 8 vCPU / 32 GiB: decode + x264 are CPU-side (NVDEC is a deferred D16 knob), so CPU
headroom is
46     what keeps the paid GPU busy. `/tmp` is **RAM-backed** ? the pulled input video lives
in memory,
47     so size --memory` for the largest expected upload (a 5.8 GB GoPro file ? 6 GiB of
tmpfs).
48
49     ## 3. Wire the dispatcher (control plane)
50     The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
51     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
52     ```bash
53     export DEPLOYMENT_PROFILE=cloud-serving
54     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
55     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
56     # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
57     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
58     ```
59     **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret
Manager secret
60     on the job), or run fully-local with --set indexing.skip_ai_handoff=true` (candidate
windows
61     emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent
62     0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail
this at
63     boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.
64
65     ### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs
66
67     The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-
PROCESS**: the
68     request returns `202` and a background thread drains the job (GPU detection dispatches
to the Job;
69     the reel stitch runs on the control plane itself). On Cloud Run that background thread
only makes
70     progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
71
72     **The canonical deploy path is the committed manifest generator** (#473) ? it bakes the
cloud-serving config.json into a startup wrapper, sets every flag below, pins
`maxScale=1`
74     (per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD_RUN_JOB`
(a
75     reserved runtime env name that gets a manifest rejected; the code default is already
right):
76     ```bash
77     python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
78     gcloud run services replace service.yaml --region $REGION # from PowerShell on Windows
79     ```
80     --edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with
`on`
81     before testers touch the service again. The flag-based equivalent (for reference ? a

```

```

manifest
82     round-trips the bash wrapper's metacharacters, flags don't on Windows):
83     ```bash
84     gcloud run deploy rally-control-plane --image <image> --region $REGION \
85     --no-cpu-throttling \           # CPU stays allocated between requests ? the drain thread
finishes
86     --min-instances 1 \           # one always-warm instance drains reliably (cheap CPU;
NOT the D7 GPU pool)
87     --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
88     --set-env-vars DEPLOYMENT_PROFILE=cloud-
serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
89     ```
90     Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the
reel never
91     finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT
contradict
92     D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-
in-depth is in
93     the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a
restart (idempotent
94     stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on
the next drain.
95     Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to
the CF proxy
96     (M9) before inviting testers. The image ships the `auth` extra (PyJWT[crypto]), so
flipping
97     `security.edge_auth_enabled` on needs no derived image ? the M9 Cf-Access JWT verify
works out of
98     the box.
99
100    ## 4. M7 acceptance (the owner runlog)
101    Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
102    `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
103    rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
104    exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
105
106    ## Gotchas
107    - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
108    + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
109    - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
110    ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
111    - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
112    real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
113    video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
114    (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
115    surfaces it as a `cloud dispatch error` job failure (not a traceback).
116    - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
117    - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`
118    fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
`job_path`;
119    an unset project would otherwise build a bogus `projects/None/...` name). Export it
(step 3) before
120    dispatching. The guard runs before the `google-cloud-run` import, so it is exercised

```

```

in CI even
121     though the SDK isn't a CI/test dependency.
122     - **Hand-rolled `jobs execute --args` with `gs://` URIs:** gcloud's default comma-split
plus its
123     alternate-delimiter syntax both bite ? repeated `--set` tokens trip the comma parser,
and a
124     `^:^\` delimiter splits the `:` inside `gs://`. Use a delimiter that appears nowhere in
the args,
125     e.g. `--args='^@@^-v@@infer@@--video-uri=gs://?'`. (The dispatcher path doesn't have
this
126     problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must
include the
127     dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu`,
128     `indexing.detector_impl=native`.
129     - **Git Bash (MSYS) on Windows mangles Unix-looking values** before gcloud sees them:
130     `--add-volume-mount ?mount-path=/gcs` became `C:/Program Files/Git/gcs`, and
131     `--command=/bin/bash` became `C:/Program Files/Git/usr/bin/bash` (the container then
exec-fails).
132     Run gcloud commands with such flags from PowerShell (`MSYS_NO_PATHCONV=1` breaks
gcloud's own
133     wrapper). PowerShell has its own trap: quote any flag value containing commas.
134     - **First execution after an image push** spends minutes in "Container image import in
progress"
135     (visible in `executions describe status.conditions`, NOT in logs) ? that's progress,
not a hang;
136     later executions reuse the imported image.
137

```

31. user (2026-07-06T17:11:30.820Z)

```

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\deploy\cloudrun\README.md
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\deploy\digitalocean\README.md
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\deploy\gcp\nightly_regression\README.md
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md

```

32. assistant (2026-07-06T17:11:32.130Z)

Perfect! Now let me check the test files to understand the dispatch mechanism better:

33. user (2026-07-06T17:11:33.020Z)

```

1     """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3     All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4     worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5     ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6     selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7     """
8
9     import json
10    import os
11    import tempfile
12
13    import pytest
14
15    from backend.compute import ComputeJobSpec, get_compute_backend
16    from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
17    from backend.config import AppConfig
18    from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout

```

```

19
20     # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).
21     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
22
23
24     class _FakeRunClient(CloudRunJobClient):
25         """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
local bucket
26         exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""
27
28         def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
29             self.calls: list[dict] = []
30             self.video_id, self.segment_count, self.returncode = (
31                 video_id,
32                 segment_count,
33                 returncode,
34             )
35
36         def run_job(self, job_name, argv, *, region, project=None):
37             self.calls.append(
38                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
39             )
40             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
41             done_uri = argv[argv.index("--done-uri") + 1]
42             outputs = os.path.join(out_uri, "outputs", self.video_id)
43             os.makedirs(outputs, exist_ok=True)
44             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
45                 json.dump(
46                     {
47                         "video_id": self.video_id,
48                         "segment_count": self.segment_count,
49                         "status": "success",
50                     },
51                     f,
52                 )
53             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
54             with open(done_uri, "w", encoding="utf-8") as f:
55                 json.dump(
56                     {
57                         "video_id": self.video_id,
58                         "returncode": self.returncode,
59                         "segment_count": self.segment_count,
60                         "status": "success",
61                     },
62                     f,
63                 )
64             return "exec-123"
65
66         def cancel_execution(self, execution):
67             raise AssertionError(
68                 "cancel must never fire on the happy path"
69             ) # pragma: no cover
70
71
72     class _NoopRunClient(CloudRunJobClient):
73         """Records the dispatch but does NO filesystem writes ? for tests that fake the
storage layer
74         directly (a gs:// out_uri has no local dir to makedirs)."""
75
76         def __init__(self):
77             self.calls: list[dict] = []
78             self.cancelled: list[str] = []
79

```

```

80     def run_job(self, job_name, argv, *, region, project=None):
81         self.calls.append(
82             {"job": job_name, "argv": list(argv), "region": region, "project": project}
83         )
84         return "exec"
85
86     def cancel_execution(self, execution):
87         self.cancelled.append(execution)
88
89
90     class _NeverDoneClient(CloudRunJobClient):
91         """Simulates a still-running execution: dispatch succeeds (returning a fully-
qualified
92         execution name) but NO done-marker is ever written, so the bucket-poll must time
out."""
93
94         EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-
slow"
95
96         def __init__(self, *, cancel_error: Exception | None = None):
97             self.calls: list[dict] = []
98             self.cancelled: list[str] = []
99             self._cancel_error = cancel_error
100
101         def run_job(self, job_name, argv, *, region, project=None):
102             self.calls.append(
103                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
104             )
105             return self.EXEC_NAME
106
107         def cancel_execution(self, execution):
108             self.cancelled.append(execution)
109             if self._cancel_error is not None:
110                 raise self._cancel_error
111
112
113     # ----- factory (default-
OFF)
114
115
116     def _cfg(compute_target="", storage_backend="local"):
117         return AppConfig.model_validate(
118             {
119                 "deployment": {"compute_target": compute_target},
120                 "storage": {"backend": storage_backend},
121             }
122         )
123
124
125     def test_factory_is_none_for_local_targets():
126         """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
127         assert get_compute_backend(_cfg("")) is None
128         assert get_compute_backend(_cfg("local_gpu")) is None
129         assert get_compute_backend(_cfg("cpu_mock")) is None
130
131
132     def test_factory_returns_cloud_run_for_cloud_target():
133         backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
134         assert isinstance(backend, CloudRunCompute)
135
136
137     # ----- real client: missing-project
guard
138

```

```

139
140 def test_real_client_rejects_missing_project():
141     """The real GoogleCloudRunJobClient must fail loudly when GOOGLE_CLOUD_PROJECT is
unset ?
142     a None/empty project would build a bogus "projects/None/..." job path (job_path
requires a
143     real str). The guard runs BEFORE the google-cloud-run import, so it is exercisable
in CI
144     (the SDK is intentionally not a test dependency)."""
145     from backend.compute.cloud_run import GoogleCloudRunJobClient
146
147     client = GoogleCloudRunJobClient()
148     for missing in (None, ""):
149         with pytest.raises(RuntimeError, match="GOOGLE_CLOUD_PROJECT"):
150             client.run_job(
151                 "rally-worker", ["infer"], region="asia-south1", project=missing
152             )
153
154
155 def test_real_client_cancel_rejects_unresolvable_execution_name():
156     """cancel_execution must fail loudly on a non-resolvable execution name ? run_job
falls back
157     to the literal "execution" when the operation metadata carries no name, and
cancelling that
158     would be a bogus API call. The guard runs BEFORE the google-cloud-run import
(mirrors the
159     project guard), so it is exercisable in CI (the SDK is intentionally not a test
dependency)."""
160     from backend.compute.cloud_run import GoogleCloudRunJobClient
161
162     client = GoogleCloudRunJobClient()
163     for bogus in ("", "execution", "exec-123"):
164         with pytest.raises(RuntimeError, match="fully-qualified"):
165             client.cancel_execution(bogus)
166
167
168 # ----- CloudRunCompute dispatch
+ poll
169
170
171 def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
172     client = _FakeRunClient(video_id="abc123", segment_count=3)
173     backend = CloudRunCompute(
174         run_client=client,
175         job_name="rally-worker",
176         region="asia-south1",
177         project="proj",
178         storage_config={},
179         policy=_FAST,
180         token="tok",
181     )
182     out = str(tmp_path / "bucket")
183     result = backend.run_infer(
184         ComputeJobSpec(
185             video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"
186         )
187     )
188
189     assert result.returncode == 0
190     assert result.video_id == "abc123"
191     assert result.outputs_uri == f"{out}/outputs/abc123/"
192     assert result.report.get("segment_count") == 3
193     # exactly one Cloud Run Job execution was triggered, in the right region/project
194     assert len(client.calls) == 1
195     assert (

```

```

196         client.calls[0]["job"] == "rally-worker"
197         and client.calls[0]["region"] == "asia-south1"
198     )
199
200
201     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
202         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
203         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
204         client = _FakeRunClient()
205         backend = CloudRunCompute(
206             run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
207         )
208         backend.run_infer(
209             ComputeJobSpec(
210                 video_uri="gs://b/v.mp4",
211                 out_uri=str(tmp_path / "bk"),
212                 segmenter="wasb_hybrid",
213                 config_overrides={"sport=tennis",},
214             )
215         )
216
217         argv = client.calls[0]["argv"]
218         assert argv[0] == "infer"
219         assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
220         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
221         # the container-inline overrides come last (win) and the user override is carried
through
222         joined = " ".join(argv)
223         assert "deployment.compute_target=local_gpu" in joined
224         assert "indexing.detector_impl=native" in joined
225         assert "sport=tennis" in joined
226         # user override precedes the inline-forcing overrides in --set order
227         assert joined.index("sport=tennis") < joined.index(
228             "deployment.compute_target=local_gpu"
229         )
230
231
232     def test_unique_token_per_call_when_not_injected(tmp_path):
233         """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
234         client = _FakeRunClient()
235         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
236         backend.run_infer(
237             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
238         )
239         backend.run_infer(
240             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
241         )
242         done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
243         done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
244         assert done1 != done2
245
246
247     def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
248         """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a None dst raises) ?
_read_json must
249         pass one, else every real cloud marker/report read silently returns {} (local hid
this via its
250         identity passthrough). Mimic a cloud backend that rejects dst=None."""

```

34. assistant (2026-07-06T17:11:34.940Z)

Excellent! Now I have all the information I need. Let me compile the findings into a

comprehensive report.

35. user (2026-07-06T17:11:35.311Z)

Chapter marked: "Investigation Complete". Continue your current work.

36. assistant (2026-07-06T17:12:03.648Z)

Findings Report: Rally-Worker Dispatch Mechanism & Custom Test Run Recipe

1. **How rally-worker learns WHICH video to process**

Answer: The worker receives the video URI via command-line arguments (`--video-uri`), NOT from env vars or a dispatch file.

Exact mechanism:

- File: `backend/worker/__main__.py:649-651` (arg parser definition)
- The worker's entrypoint is `python -m backend.worker infer --video-uri <uri> --out-uri <prefix> [--segmenter ...] [--done-uri ...]`
- `--video-uri` is **REQUIRED** and can be any URI scheme: `local://`, bare path, `s3://`, or `gs://`
- Example: `python -m backend.worker infer --video-uri gs://khelsutra-rally-serving/videos/uploads/<uploadId>/<name>.MP4 --out-uri gs://khelsutra-rally-serving --segmenter wasb_hybrid`

Payload format: None?arguments are flat command-line strings. There is **no** JSON dispatch file in `_dispatch/` for input configuration; that directory (`gs://.../_dispatch/`) is reserved **ONLY** for output completion markers.

2. **How an execution is actually launched**

Answer: The control plane (rally-control-plane service) triggers the job via the Cloud Run Admin API using the `google-cloud-run` SDK.

Exact mechanism:

- File: `backend/compute/cloud_run.py:84-123` (`GoogleCloudRunJobClient.run_job`)
- **Command-like equivalent:**

```
gcloud run jobs execute rally-worker \
  --region asia-south1 \
  --update-env-vars ... \
  --args=infer,--video-uri,gs://...,--out-uri,gs://...,...
```

But the **real code path** uses the SDK directly:

```
python
from google.cloud import run_v2
client = run_v2.JobsClient()
name = client.job_path(project, region, job_name) # e.g. "projects/gen-lang-
client-0306026826/locations/asia-southeast1/jobs/rally-worker"
overrides = run_v2.RunJobRequest.Overrides(
    container_overrides=[
        run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
    ]
)
op = client.run_job(request=run_v2.RunJobRequest(name=name, overrides=overrides))
```

This returns an execution ID (e.g., `projects/p/locations/r/jobs/rally-worker/executions/exec-xyz`).

- **Triggering flow:**

1. Control plane API `/api/process` receives a video upload or URI.
2. `backend/orchestration.py:564` calls `compute.run_infer(spec)`.

3. CloudRunCompute builds the argv from the spec and calls ``self._client.run_job(...)``.
4. The container runs: ``python -m backend.worker infer <argv>`` on an L4 GPU.
5. Worker writes completion marker to ``gs://.../_dispatch/<token>.done``.
6. Control plane bucket-polls the marker and reads ``outputs/<video_id>/report.json``.

3. ****Environment variables the job reads at runtime****

****Worker-side env vars**** (set in Dockerfile or job env):

Env Var	Source	Purpose
<code>`WASB_PYTHON`</code>	Dockerfile:80	Path to conda Python for WASB shuttle detector (e.g., <code>`/opt/conda/envs/wasb/bin/python`</code>)
<code>`WASB_REPO_DIR`</code>	Dockerfile:81	Path to WASB repo (e.g., <code>`/opt/models/WASB-SBDT`</code>)
<code>`WASB_WEIGHTS`</code>	Dockerfile:82	Path to pretrained weights (mounted via GCSFuse: <code>`/gcs/models/wasb_badminton_best.pth.tar`</code>)
<code>`RALLY_DETECTOR_IMPL`</code>	Dockerfile:83	Detector implementation: <code>`native`</code> (on GPU; runs WASB via subprocess) or <code>`stub`</code> (CPU mock)
<code>`PYTHONUNBUFFERED`</code>	Dockerfile:1, :79	Set to 1 to stream logs live instead of buffering until exit
<code>`DEPLOYMENT_PROFILE`</code>	gen_service_yaml.py:130	Baked in control-plane manifest: <code>`cloud-serving`</code>
<code>`CLOUD_RUN_REGION`</code>	gen_service_yaml.py:133	e.g., <code>`asia-southeast1`</code>
<code>`GOOGLE_CLOUD_PROJECT`</code>	gen_service_yaml.py:135	e.g., <code>`gen-lang-client-0306026826`</code>
<code>`RALLY_APP_HOME`</code>	gen_service_yaml.py:100 (control plane only)	Path to config.json directory (<code>`/tmp/apphome`</code>)

****Control-plane env vars**** (for the dispatcher to find the job):

Env Var	Default	Purpose
<code>`CLOUD_RUN_JOB`</code>	<code>`"rally-worker"`</code>	Job name (backend/compute/___init__.py:79)
<code>`CLOUD_RUN_REGION`</code>	<code>`"asia-south1"`</code>	Job region (backend/compute/___init__.py:80)
<code>`GOOGLE_CLOUD_PROJECT`</code>	(unset)	GCP project; **REQUIRED** for real client (backend/compute/___init__.py:81)

4. ****Concrete recipe to run rally-worker on a custom downsampled video****

****Step 1: Upload your custom video to GCS****

```

`bash
gsutil cp your_downsampled_video.mp4 \
gs://khelsutra-rally-serving/videos/uploads/my-test-id/my-video.MP4
`

```

****Step 2: Trigger the job manually via gcloud (one-off test)****

****For a non-interactive manual dispatch:****

```

`bash
export SERVING_BUCKET="gs://khelsutra-rally-serving"
export PROJECT="gen-lang-client-0306026826"
export REGION="asia-southeast1"

gcloud run jobs execute rally-worker \
  --region $REGION \
  --args='^@@^infer@--video-uri="$SERVING_BUCKET"/videos/uploads/my-test-id/my-
video.MP4@--out-
uri="$SERVING_BUCKET"@@--segmenter=wasb_hybrid@@--set=indexing.skip_ai_handoff=true'
`

```

****OR use the Python API directly**** (if you have ``google-cloud-run`` installed):

```

`python

```

```

from google.cloud import run_v2

client = run_v2.JobsClient()
job_name = client.job_path(
    "gen-lang-client-0306026826", # project
    "asia-southeast1",           # region
    "rally-worker"
)

overrides = run_v2.RunJobRequest.Overrides(
    container_overrides=[
        run_v2.RunJobRequest.Overrides.ContainerOverride(
            args=[
                "infer",
                "--video-uri", "gs://khelsutra-rally-serving/videos/uploads/my-test-id/my-
video.MP4",
                "--out-uri", "gs://khelsutra-rally-serving",
                "--segmenter", "wasb_hybrid",
                "--set", "indexing.skip_ai_handoff=true",
            ]
        )
    ]
)

op = client.run_job(request=run_v2.RunJobRequest(name=job_name, overrides=overrides))
execution_name = op.metadata.name # e.g., projects/p/locations/r/jobs/rally-
worker/executions/exec-xyz
print(f"Triggered execution: {execution_name}")
...

```

Step 3: Outputs

The worker writes results to:

```

...
gs://khelsutra-rally-serving/outputs/<video_id>/
??? intervals.csv          # Rally segments (start, end, shot_count, ending_reason)
??? report.json            # Metadata (video_id, status, segment_count, segments)
??? (optional) other artifacts

gs://khelsutra-rally-serving/_dispatch/<token>.done
??? Completion marker: {"video_id": "...", "returncode": 0, "segment_count": N, "status":
"success"}
...

```

Where ``<video_id>`` is the **MD5 hash of the video file bytes** (computed by the worker at runtime; same as the control plane's submit-time hash via `gsutil stat -m`).

Step 4: Monitor

```
``bash
```

Watch the job execution logs

```
gcloud run jobs executions log <execution-name> \
  --region asia-southeast1
```

Verify outputs exist

```
gsutil ls -h gs://khelsutra-rally-serving/outputs/
gsutil cat gs://khelsutra-rally-serving/outputs/<video_id>/report.json | jq .
...

```

Key Technical Details

```

**Dispatch token (`_dispatch/<token>.done`):**
- File: `backend/compute/cloud_run.py:186-187`
- Each execution gets a unique `token` (UUID or injected for tests).
- Prevents concurrent jobs to the same bucket from colliding on markers.

**Container forced-inline overrides:**
- File: `backend/compute/cloud_run.py:48-50`
- The dispatched container always receives:
  - `--set deployment.compute_target=local_gpu` (disables re-dispatch)
  - `--set indexing.detector_impl=native` (uses WASB on L4)
- User `--set` overrides come first in argv, so they can be overridden by the inlining flags.

**Completion poll:**
- File: `backend/orchestration.py:564-567`
- Control plane bucket-polls `_dispatch/<token>.done` every few seconds (default
  `ExecutionPolicy.poll_every_n_sec`).
- Timeout: `deployment.remote_poll_max_wait_sec` (default 1800s, should ? job's `--task-timeout`
  of 3600s).

**Config baking (control plane only):**
- File: `deploy/cloudrun/gen_service_yaml.py:92-101`
- The manifest base64-encodes `config.json` into the startup wrapper so every config pin
  survives redeployments (wasb_hybrid segmenter, 24 MB max upload part).

```

Summary Command for Your Test

****To process your downsampled video on the production rally-worker:****

```

```bash
gsutil cp my_downsampled.mp4 \
 gs://khesutra-rally-serving/videos/uploads/test-run-001/downsampled.MP4

gcloud run jobs execute rally-worker \
 --region asia-southeast1 \
 --args='^@@^infer@@--video-uri=gs://khesutra-rally-serving/videos/uploads/test-
run-001/downsampled.MP4@@--out-uri=gs://khesutra-rally-
serving@@--segmenter=wasb_hybrid@@--set=indexing.skip_ai_handoff=true'
```

```

Results appear in `gs://khesutra-rally-serving/outputs/<MD5>/` shortly after execution completes.